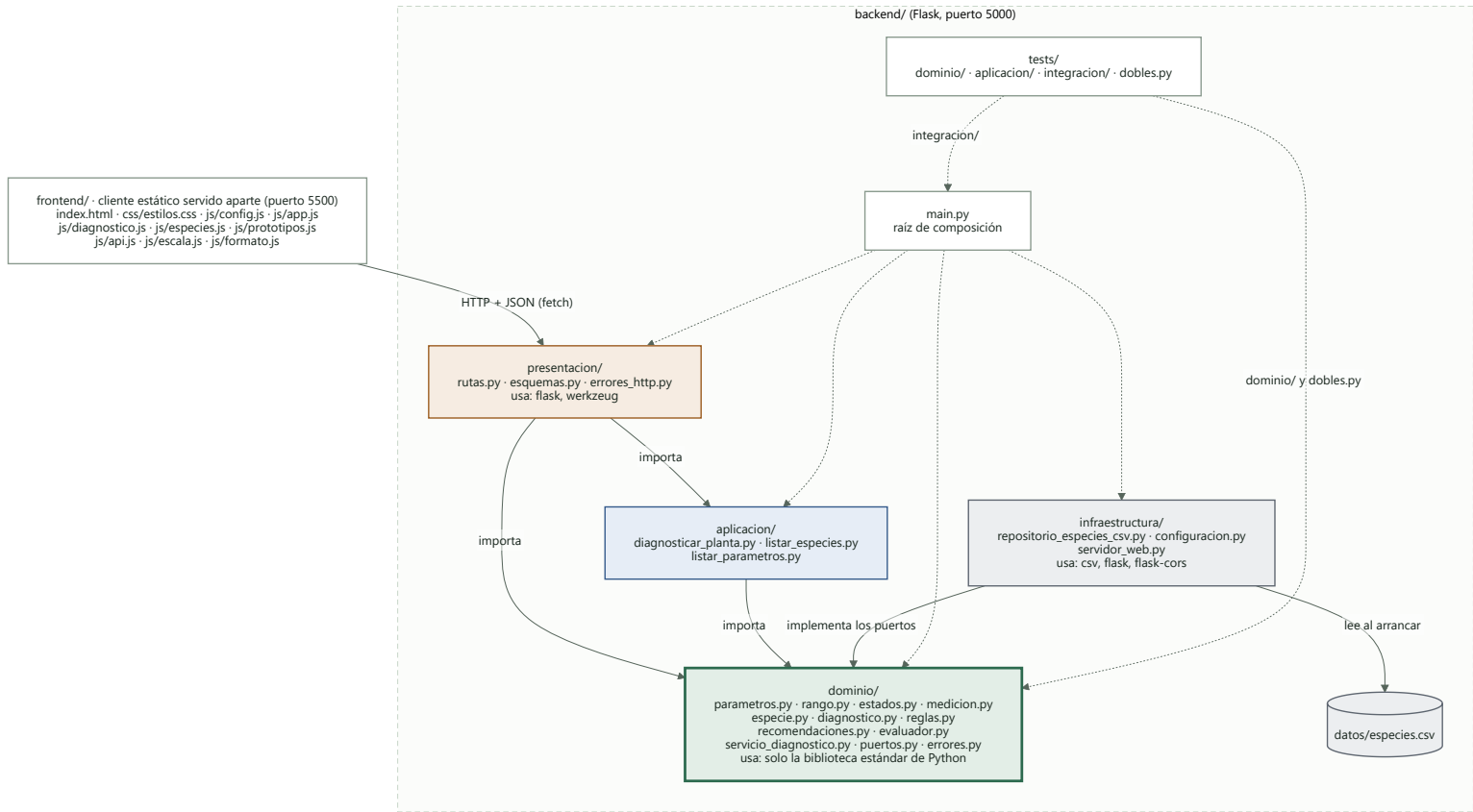


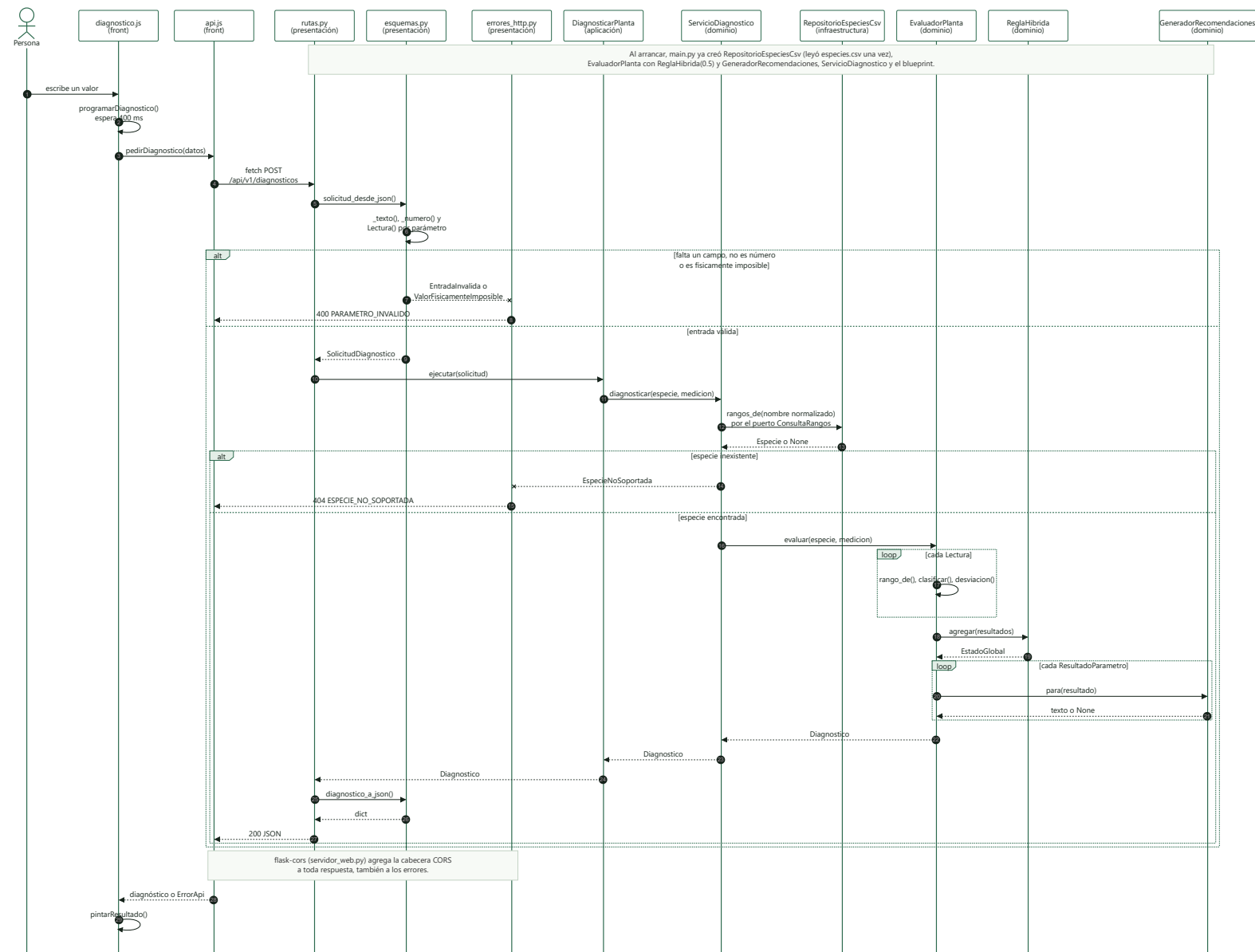
El sistema es el núcleo de la matera inteligente: una API que recibe una especie y una medición de humedad del sustrato, luz y temperatura, y devuelve el estado de la planta. El backend (Python 3.11 y Flask) está dividido en cuatro capas: presentación, aplicación, dominio e infraestructura. La tabla de referencia se consulta por puertos declarados en el dominio. El front es un cliente estático servido aparte que usa `fetch`. La API expone `GET /api/v1/especies`, `GET /api/v1/parametros` y `POST /api/v1/diagnosticos`, y todo error responde JSON con la forma `{error, mensaje, detalle}`. Hay 79 pruebas: 47 del dominio, que corren sin servidor ni CSV, 3 de aplicación y 29 de integración. Las referencias `archivo:línea` son relativas a `backend/` salvo que digan otra cosa.

1. Diagrama de paquetes



Cada caja es una carpeta real del repositorio con todos sus archivos (sin los `__init__.py`). Las flechas continuas son `import` del código de producción, sacados de los archivos, no dibujados a mano. Ninguna sale de `dominio/`, que solo usa la biblioteca estándar; `tests/dominio/test_arquitectura.py:34` falla si eso cambia. La flecha de infraestructura a dominio es la implementación de los puertos: la dependencia apunta hacia el dominio (RA5). Las punteadas son `main.py`, la raíz de composición y único archivo que instancia clases concretas, y las pruebas. El front solo conoce la API por HTTP (RA2).

2. Diagrama de secuencia: una petición de diagnóstico



Pasos 1 a 4: el front espera 400 ms después de la última tecla y envía la petición. Pasos 5 y 6: `esquemas.py` convierte el JSON en `Lectura` y `Medicion`; desde `presentacion/esquemas.py:30` el dato ya no sabe que llegó por HTTP (RA6), y los límites físicos se validan en el propio `Lectura`. Paso 12: el servicio pide los rangos por el puerto `ConsultaRangos`; el CSV ya se había leído al arrancar. Los errores 400 y 404 salen de `errores_http.py` con el cuerpo uniforme (RF6).

3. Responsabilidades por capa

Capa	Qué hace	Qué tiene prohibido	De qué depende
Presentación presentacion/	Expone <code>/api/v1</code> (<code>rutas.py</code>). Valida la forma de la entrada (campo presente, valor numérico) y la convierte en objetos del dominio (<code>esquemas.py</code>). Traduce excepciones a códigos HTTP con cuerpo uniforme, incluidos 404, 405 y 500 (<code>errores_http.py</code>).	Clasificar o decidir estados; leer el CSV; generar HTML (RA1); pasarle a la aplicación un <code>request</code> o un <code>dict</code> (RA6).	Aplicación (casos de uso y <code>SolicitudDiagnostico</code>), dominio (objetos, errores y <code>PARAMETROS</code>), flask y <code>werkzeug</code> .
Aplicación aplicacion/	Casos de uso <code>DiagnosticarPlanta</code> , <code>ListarEspecies</code> (ordena por nombre) y <code>ListarParametros</code> . Es el lugar donde entrarían el histórico, los usuarios o los eventos, sin tocar las reglas.	Conocer HTTP, Flask o el CSV; tener reglas de negocio propias.	Dominio.
Dominio dominio/	Objetos: <code>Parametro</code> , <code>Rango</code> , <code>Lectura</code> , <code>Medicion</code> , <code>Especie</code> y <code>Diagnostico</code> . Reglas: clasificar en BAJO, OPTIMO o ALTO y medir la desviación (<code>rango.py</code>), límites físicos (<code>medicion.py</code>), estado global (<code>reglas.py</code>), recomendaciones, <code>EvaluadorPlanta</code> y <code>ServicioDiagnostico</code> . Declara los puertos <code>ConsultaRangos</code> y <code>CatalogoEspecies</code> .	Importar flask, csv, http, json o cualquier otra capa (RA4). Lo comprueba <code>tests/dominio/test_arquitectura.py</code> .	Solo la biblioteca estándar de Python.
Infraestructura infraestructura/	<code>RepositorioEspeciesCsv</code> implementa los dos puertos y carga el CSV al arrancar, fallando con un mensaje claro si el archivo está mal. <code>Configuracion</code> lee variables de entorno. <code>servidor_web.py</code> arma Flask con JSON y CORS limitado a los orígenes del front (RA7).	Reglas de negocio; rutas; decidir estados.	Dominio (puertos y objetos), csv, flask y flask-cors.
Raíz de composición main.py	Crea las clases concretas y las conecta: repositorio, evaluador con <code>ReglaHibrida(umbral=0.5)</code> , servicio, casos de uso, blueprint y aplicación.	Lógica de negocio o de HTTP.	Las cuatro capas.
Front frontend/	Diagnóstico en vivo y catálogo de especies con datos reales de la API. Historial, materias y vinculación del sensor son maquetas marcadas como fuera de alcance, pero también piden sus estados a la API.	Escribir a mano rangos, umbrales, límites, parámetros o textos de recomendación; decidir estados.	La API por HTTP (<code>frontend/js/api.js</code>).

4. Justificación SOLID

S • Responsabilidad única

La clase que decide el estado no lee el CSV ni arma la respuesta HTTP. `EvaluadorPlanta.evaluar()` (`dominio/evaluador.py:20`) clasifica, aplica la regla y junta las recomendaciones. `ReglaHibrida` (`dominio/reglas.py:19`) solo agrega estados. El único `import csv` del proyecto está en `infraestructura/repositorio_especies_csv.py:7`. La respuesta la arma `presentacion/esquemas.py:34` y el controlador (`presentacion/rutas.py:27-31`) solo convierte, ejecuta y responde.

Sin aplicar: si el evaluador leyera el CSV y armara el JSON, pasar a PostgreSQL o cambiar el formato de respuesta obligaría a modificar la clase que decide el estado, y su prueba (`tests/dominio/test_evaluador.py:20`) necesitaría archivo y servidor.

Tensión: `esquemas.py` tiene dos razones para cambiar: el contrato de entrada (: 25) y el de salida (: 34). Quedaron juntos porque ambos son el contrato de la API v1; si crecen, se separan.

O · Abierto/cerrado

`ReglaAgregacion` (`dominio/reglas.py:13`) es la abstracción. `EvaluadorPlanta` la recibe por constructor (`dominio/evaluador.py:16`) y solo llama `agregar()` (`:25`). Cambiar de regla es escribir una clase nueva y cambiar `main.py:27`; `ReglaConteo` (`reglas.py:41`) lo demuestra en `tests/dominio/test_servicio_diagnostico.py:44`. El evaluador tampoco nombra parámetros: recorre `medicion.lecturas` (`evaluador.py:21`). Presentación e infraestructura recorren el catálogo `PARAMETROS` (`dominio/parametros.py:29`, `esquemas.py:30`, `repositorio_especies_csv.py:44`). `tests/dominio/test_evaluador.py:46` evalúa un pH sin tocar el evaluador.

Sin aplicarlo: con tres campos fijos o un `if` por parámetro, agregar el pH obligaría a modificar `Medicion`, `Especie`, el evaluador, el lector del CSV y el DTO.

Tensión: agregar el pH no toca ninguna lógica, pero sí modifica datos:

- el catálogo (`parametros.py:29`);
- las columnas del CSV y la tabla de ejemplo de `tests/integracion/test_api.py`;
- opcionalmente, los textos (`recomendaciones.py:6`) y la etiqueta del front (`frontend/js/formato.js:7`).

L · Sustitución de Liskov

`ServicioDiagnostico` recibe cualquier `ConsultaRangos` (`dominio/servicio_diagnostico.py:17`). Según el contrato (`dominio/puertos.py:16-19`), `rangos_de()` devuelve una `Especie` o `None`. Lo cumplen el repositorio CSV (`infraestructura/repositorio_especies_csv.py:26`) y el doble en memoria (`tests/dobles.py:24`): las reglas pasan con el doble en las pruebas de dominio y con el CSV en `tests/integracion/test_api.py:64`.

Sin aplicarlo: si el servicio preguntara por la clase concreta o usara algo propio del CSV, como la ruta del archivo, el doble no serviría y las pruebas de dominio tendrían que leer el archivo.

Tensión: la precondition "nombre normalizado" solo está en el docstring (`puertos.py:19`). El CSV vuelve a normalizar (`repositorio_especies_csv.py:27`) y el doble no (`tests/dobles.py:24`). Con el servicio actual se comportan igual, pero una llamada directa con " Potos " daría resultados distintos.

I · Segregación de interfaces

El dominio no depende de un repositorio genérico, sino de dos puertos de un solo método: `ConsultaRangos` (`dominio/puertos.py:12`) para diagnosticar y `CatalogoEspecies` (`:23`) para listar. `ServicioDiagnostico` conoce solo el primero (`servicio_diagnostico.py:7`) y `ListarEspecies` solo el segundo (`aplicacion/listar_especies.py:3`). Una misma clase implementa ambos (`repositorio_especies_csv.py:21`): la segregación es para los clientes, no para quien implementa.

Sin aplicarlo: el doble de las pruebas y cualquier adaptador de solo lectura tendrían que implementar métodos que nadie usa.

Tensión: `ListarParametros` (`aplicacion/listar_parametros.py:12`) recibe el catálogo directamente (`main.py:33`), sin puerto, porque los parámetros son código del dominio y no un dato externo. Si pasaran a una base de datos, haría falta un tercer puerto.

D · Inversión de dependencias

Es la formalización de RA5:

- `ServicioDiagnostico` importa el puerto, no el CSV (`servicio_diagnostico.py:7`).
- `RepositorioEspeciesCsv` importa los puertos y los implementa (`repositorio_especies_csv.py:13` y `:21`); la dependencia va de infraestructura hacia el dominio.
- La única línea que elige la clase concreta es `main.py:24`, y la inyección ocurre en `main.py:28`.
- Las rutas reciben los casos de uso (`presentacion/rutas.py:11`) y `crear_app()` recibe el blueprint (`infraestructura/servidor_web.py:8`).

Sin aplicarlo: si el servicio creara el repositorio CSV por dentro, pasar a PostgreSQL cambiaría el dominio y sus pruebas leerían el archivo. Se incumplirían RA4 y RA5.

Tensión: `DiagnosticarPlanta` depende de la clase concreta `ServicioDiagnostico` (`aplicacion/diagnosticar_planta.py:6` y `:24`), no de una abstracción. Se aceptó porque ambas capas son internas y estables.

5. Plan de evolución

5.1 Las mediciones llegan por MQTT desde un ESP32

Se agrega	<code>infraestructura/mqtt/cliente.py</code> : conexión al broker y suscripción al tópico de mediciones (<code>paho-mqtt</code>). <code>presentacion/mqtt/mensajes.py</code> : convierte el mensaje en <code>SolicitudDiagnostico</code> creando <code>Lectura</code> y <code>Medicion</code> , igual que <code>esquemas.py</code> con el JSON; los límites físicos se validan solos. Opcional: publicar el diagnóstico en otro tópico.
Se modifica	<code>main.py</code> (arranca el cliente y le pasa el mismo <code>DiagnosticarPlanta</code>), <code>infraestructura/configuracion.py</code> (dirección del broker) y <code>requirements.txt</code> .
No se toca	Todo <code>dominio/</code> y <code>aplicacion/</code> ; <code>rutas.py</code> , <code>esquemas.py</code> y <code>errores_http.py</code> ; el repositorio CSV; el front.

El ESP32 es otra entrada al mismo caso de uso. El dato deja de saber de MQTT en `mensajes.py`, como hoy deja de saber de HTTP en `esquemas.py:30`.

5.2 La tabla de referencia pasa a una base de datos relacional

Se agrega	<code>infraestructura/repositorio_especies_sql.py</code> , que implementa <code>ConsultaRangos</code> y <code>CatalogoEspecies</code> . <code>datos/esquema.sql</code> y <code>datos/carga.sql</code> , con las tablas <code>especie</code> y <code>rango_optimo(especie, parametro, minimo, maximo)</code> : una fila por parámetro, para que agregar el pH no cambie el esquema. Pruebas de integración del repositorio nuevo.
Se modifica	<code>main.py:24</code> (una línea: la clase concreta), <code>configuracion.py</code> (cadena de conexión) y <code>requirements.txt</code> (driver).
No se toca	<code>dominio/</code> , <code>aplicacion/</code> , <code>presentacion/</code> , el front y las 50 pruebas de dominio y aplicación.

5.3 Aparecen usuarios, cada uno con varias plantas

Se agrega	Dominio: módulo nuevo <code>usuarios.py</code> (<code>Usuario</code> ; <code>Planta</code> con dueño, especie y apodo) y los puertos <code>RepositorioPlantas</code> e <code>HistorialMediciones</code> . Aplicación: <code>RegistrarPlanta</code> , <code>ListarPlantas</code> y <code>DiagnosticarPlantaDeUsuario</code> , que busca la planta, toma su especie, llama al <code>ServicioDiagnostico</code> existente y guarda la medición. Presentación: <code>rutas_plantas.py</code> con autenticación por token. Infraestructura: repositorios SQL. Front: la pantalla Materas deja de ser maqueta.
Se modifica	<code>main.py</code> (conectar lo nuevo); <code>infraestructura/servidor_web.py</code> (hoy <code>crear_app()</code> registra un solo blueprint y tendría que aceptar varios); <code>frontend/js/api.js</code> y <code>prototipos.js</code> .
No se toca	Los módulos actuales del dominio (evaluador, reglas, rango, parámetros, puertos), <code>DiagnosticarPlanta</code> , el repositorio CSV y las rutas actuales.

El diagnóstico sigue dependiendo solo de la especie. El usuario decide sobre qué planta se consulta y a quién pertenece la medición.

5.4 Se agrega gamificación: puntos, rachas y niveles

Se agrega	Módulo de dominio propio <code>gamificacion.py</code> (<code>Progreso</code> , <code>Racha</code> y <code>Nivel</code> , con reglas como puntos por día SALUDABLE) y el puerto <code>RepositorioProgreso</code> . En aplicación, <code>RegistrarProgreso</code> y un decorador <code>DiagnosticarConProgreso</code> , que envuelve a <code>DiagnosticarPlanta</code> y suma puntos con el <code>Diagnostico</code> ya calculado. Rutas y pantalla de progreso.
Se modifica	<code>main.py</code> (envuelve el caso de uso con el decorador) y el front.
No se toca	El dominio de la planta y <code>DiagnosticarPlanta</code> : el decorador lo extiende sin modificarlo.

No va dentro del dominio de la planta porque cambia por otras razones: el juego responde a decisiones de producto y el diagnóstico, a la botánica. Tiene su propio dominio, que depende del `Diagnostico` y no al revés. Requiere el escenario 5.3, porque los puntos son de un usuario.

6. Decisiones y alternativas descartadas

Decisión	Alternativa descartada	Por qué se descartó
Regla híbrida para el estado global (<code>reglas.py:19</code>). SALUDABLE si todo está en rango; CRITICO si hay dos o más fuera o alguno se desvía más de 0,5 veces el ancho de su rango; EN_RIESGO en el resto.	Solo contar parámetros fuera, o solo medir cuánto se desvían.	Contando, una humedad de 1 % con lo demás bien quedaba EN_RIESGO. Midiendo solo la desviación, varios problemas leves no se sumaban. La híbrida cubre ambos casos. <code>ReglaConteo</code> se conservó para demostrar el cambio de regla.
Parámetros genéricos: <code>Medicion</code> es una lista de <code>Lectura(parametro, valor)</code> .	Tres campos fijos: humedad, luz y temperatura.	Con campos fijos, agregar el pH modificaría el evaluador, la especie, el lector del CSV y el DTO (OCP).
Límites físicos en el dominio: una <code>Lectura</code> imposible no se puede crear (<code>dominio/medicion.py:20</code>).	Validarlos solo en el DTO HTTP.	Cada entrada nueva, como MQTT, tendría que repetir la validación. En el dominio ninguna medición imposible llega a existir.
El servicio de dominio consulta el puerto (<code>servicio_diagnostico.py:22</code>).	Que el caso de uso consulte el puerto y el dominio no dependa de ninguna abstracción.	"Especie no soportada" es una regla de negocio, y así las pruebas de dominio usan dobles del puerto, como pide el enunciado. Costo: hoy la capa de aplicación es delgada.
Dos puertos de un método (<code>puertos.py:12</code> y <code>:23</code>).	Un repositorio de especies con varios métodos.	Cada cliente depende solo de lo que usa, y los dobles de prueba son mínimos (ISP).
El front no decide nada: estados, rangos, parámetros y límites llegan de la API (<code>GET /api/v1/parametros</code>).	Copiar en JavaScript la regla, los rangos y los textos, como hacía el prototipo de diseño.	Duplicaría reglas en la vista, que es el error que encontramos en el ejercicio de clase, e incumpliría RF5. Por eso se agregaron el endpoint de parámetros y el nombre científico en la API.
Tabla de referencia en CSV con fuentes citadas (README).	SQLite desde el inicio.	Menos piezas para este corte. Gracias a RA5, el cambio a base de datos es una clase nueva (sección 5.2).